

Game Theory Algorithms Problem Set

*Instructor: Jing Chen**Le Gai 2025312381*

1. Chapter 1, Problem 3: TV schedules

Solution: (b) Give an example of a set of TV shows and ratings for which there is no stable pair of schedules.

There is not always a stable pair of schedules. Suppose Network A has two shows $\{a_1, a_2\}$ with ratings 20 and 40; and Network B has two shows $\{b_1, b_2\}$ with ratings 10 and 30. Each network can reveal one of two schedules.

- If in the resulting pair, a_1 is paired against b_1 (and a_2 against b_2), Network A wins both slots ($20 > 10$ and $40 > 30$). In this case, Network B will want to switch the order of the shows in its schedule to pair b_2 against a_1 . This way, $30 > 20$, so Network B will win one slot rather than none.
- If in the resulting pair, a_1 is paired against b_2 (and a_2 against b_1), Network B wins the first slot ($30 > 20$) and Network A wins the second ($40 > 10$). In this case, Network A will want to switch the order of the shows in its schedule back to pairing a_2 against b_2 and a_1 against b_1 , so that it will win two slots rather than one.

Because both possible configurations lead to one of the networks wanting to unilaterally deviate to improve its winnings, there is no stable pair of schedules in this instance.

2. Chapter 1, Problem 4: Extension of the Gale-Shapley algorithm

Proof and Algorithm:

We can solve this problem by using a many-to-one extension of the **Hospital-proposing Gale-Shapley algorithm**.

Algorithm Description:

1. Initially, all students are unassigned, and the number of available positions for each hospital equals its total capacity.
2. While there exists a hospital h that has at least one available position and has not yet offered a position to every student on its preference list:
 - Hospital h offers a position to its most preferred student s to whom it has not yet made an offer.
 - For student s :
 - If s is currently unassigned, s tentatively accepts the offer from h . h 's available positions decrease by 1.
 - If s has already tentatively accepted an offer from another hospital h' , s compares h and h' . If s prefers h to h' , s rejects h' and tentatively accepts h (h 's positions decrease by 1, h' 's positions increase by 1). If s prefers h' to h , s rejects h .
3. When no hospital can make a valid offer, the algorithm terminates, and the current tentative matching becomes the final assignment.

Proof of Stability: We need to prove that neither of the two types of instability defined in the problem can arise.

- **First type of instability does not exist:** Suppose there is a student s assigned to h , an unassigned student s' , and h prefers s' to s . Because hospitals make offers in decreasing order of their preferences, and h prefers s' to s , h must have made an offer to s' **before** making an offer to s . Since s' ends up completely unassigned, it means s' never received any offers (once a student receives an offer, they remain assigned to some hospital). This contradicts the fact that h offered a position to s' . Thus, the first type of instability cannot occur.
- **Second type of instability does not exist:** Suppose s is assigned to h , s' is assigned to h' , h prefers s' to s , and s' prefers h to h' . Similarly, since h prefers s' over s , h must have made an offer to s' before offering to s . Since s' is not assigned to h in the end, s' must have rejected h 's offer (in favor of a better hospital). This means that the hospital h' that s' eventually matches with must be strictly preferred by s' over h (i.e., $h' \succ_{s'} h$). But this contradicts the assumption that s' prefers h to h' . Thus, the second type of instability cannot occur.

This proves that the extended algorithm will always find a stable assignment.

3. Chapter 1, Problem 6: Ship maintenance

Proof and Algorithm:

This problem can be solved by reducing it to the **Stable Matching Problem**.

Let S be the set of n ships, and P be the set of n ports. We need to find a unique truncation port for each ship. We define special "preference lists" for ships and ports:

- **Ship's preferences:** Ship S_i prefers ports that it visits **earlier**. That is, S_i most prefers to truncate at its 1st scheduled port, then its 2nd, and least prefers its last scheduled port.
- **Port's preferences:** Port P_j prefers ships that arrive **later**. That is, P_j most prefers the ship that visits it on the last day of the month to be the one that stays for maintenance, and least prefers the ship that visits it on the first day.

Algorithm Description: Run the classic Gale-Shapley algorithm (either ship-proposing or port-proposing) using these defined preference lists. Since the preference lists are complete and strict, the algorithm will output a perfect stable matching. Truncate each ship's schedule at its matched port.

Proof of no conflicts: Suppose a conflict occurs under this matching, violating condition ($\dagger$). This means there are two ships, S and S' , that are in the same port P on the same day d^* . We know the original schedules had no conflicts. The only way this conflict can happen is if one ship (say S) arrives at port P on day d and stays there for maintenance, and the other ship S' sails into port P as part of its regular schedule on a later day d' ($d' > d$). Since S' visits P as part of its regular schedule, its own truncation port P' must be scheduled **after** its visit to P . That means S' visits P on day d' , which is **earlier** than the day d'' it visits P' ($d' < d''$).

Now let's check the preferences:

1. For port P : It is visited by S on day d , and by S' on day d' . Since $d' > d$ (S' arrives later), according to our defined port preferences, P prefers S' to S ($S' \succ_P S$).
2. For ship S' : It visits P on day d' and P' on day d'' . Since $d' < d''$ (P is visited earlier), according to our defined ship preferences, S' prefers P to P' ($P \succ_{S'} P'$).

Combining these two points, ship S' and port P form a **blocking pair**! But this contradicts the fact that the output of the Gale-Shapley algorithm is a "stable matching". Therefore, the assumed conflict can never happen. The set of truncations will always satisfy condition ($\dagger$).

4. Chapter 2, Problem 1: Worst Case

Proof: For a minimax tree $T_{d,k}$ of height $2k$ and branching factor d (alternating AND/OR nodes), we need to show that there exists an instance that forces any deterministic evaluation algorithm to read all d^{2k} leaves.

We can construct this worst-case scenario using an **Adversary Argument**. The adversary dynamically responds to the algorithm's queries to the leaves during execution:

- For any OR node that needs to evaluate to 0 (False), the adversary must ensure the algorithm checks all d children (which all return 0) so no short-circuiting can occur.
- For any AND node that needs to evaluate to 1 (True), the adversary must ensure the algorithm checks all d children (which all return 1) so no short-circuiting can occur.
- To prevent the algorithm from short-circuiting in other cases:
 - For an OR node whose target value is 1, the adversary intentionally makes the first $d - 1$ queried subtrees evaluate to 0. Only when the algorithm queries the d -th subtree does the adversary return 1. This forces the algorithm to explore all branches.
 - For an AND node whose target value is 0, the adversary intentionally makes the first $d - 1$ queried subtrees evaluate to 1. Only when the d -th subtree is queried does it return 0. This also forces the algorithm to explore all branches.

By recursively applying this strategy top-down, the adversary ensures that the algorithm can never achieve an early termination (short-circuit) at any internal node while determining the root's value. Consequently, any deterministic algorithm has no choice but to probe all exactly d^{2k} leaves.

5. Chapter 2, Problem 3: Evaluation Problem

(a) **Worst-case for deterministic algorithms:**

Proof: The tree has height h , branching factor 3, and internal nodes use a **majority vote**. Total leaves $n = 3^h$. Using an adversary argument: For any internal node, the algorithm needs to know that at least 2 children agree to determine the majority and short-circuit the 3rd subtree. The adversary's strategy is to ensure that whenever the algorithm queries the first two subtrees of a node, they return **opposite** values (one 0,

one 1). This results in a 1-to-1 tie, making it impossible for the algorithm to determine the majority. Thus, the algorithm is **forced** to evaluate the 3rd subtree. Since the adversary applies this delaying tactic at every internal node, the algorithm cannot skip evaluating any subtree and must read all $n = 3^h$ leaves.

(b) Expected number of leaves read by the randomized algorithm:

Proof: Let $E(h)$ be the expected number of leaves read for a tree of height h . To maximize the algorithm's reads, the adversary's only effective strategy is to place two majority values and one minority value at each node (e.g., 0, 0, 1). (If it were 0, 0, 0, picking 2 randomly guarantees agreement, leading to fewer reads). Under the worst-case distribution (2 same, 1 different), the algorithm picks 2 subtrees randomly without replacement:

- The probability of picking 2 subtrees that **agree** is $\frac{1}{3}$ (since only 1 out of $\binom{3}{2} = 3$ pairs agree). In this case, the 3rd subtree is not read.
- The probability of picking 2 subtrees that **disagree** is $\frac{2}{3}$. In this case, all 3 subtrees must be read.

Thus, the expected relationship between levels is:

$$E(h) \leq \left(\frac{1}{3} \times 2 + \frac{2}{3} \times 3 \right) E(h-1) = \left(\frac{2}{3} + 2 \right) E(h-1) = \frac{8}{3} E(h-1)$$

For the leaf nodes at the bottom, $E(0) = 1$. Solving the recurrence gives $E(h) \leq \left(\frac{8}{3}\right)^h$. Substituting tree height h with the number of leaves n : $h = \log_3 n$.

$$E(h) \leq \left(\frac{8}{3}\right)^{\log_3 n} = n^{\log_3(8/3)}$$

Calculating the exponent: $\log_3(8/3) = \log_3 8 - \log_3 3 \approx 1.8927 - 1 = 0.8927$. Since $0.8927 \leq 0.9$, the expected number of leaves read is:

$$E(h) \leq n^{0.8927} \leq n^{0.9}$$

The proof is complete.

6. Chapter 2, Problem 5: Minimax

Algebraic Proof:

Let the left side value be $v_1 = \max_q \min_i \sum_{j=1}^n a_{ij} q_j$.

Let the right side value be $v_2 = \min_p \max_j \sum_{i=1}^m p_i a_{ij}$.

Let the distribution $q^* = (q_1^*, \dots, q_n^*)$ be the one that maximizes the left side. By definition of the maximin, for all $i \in \{1, \dots, m\}$:

$$\sum_{j=1}^n a_{ij} q_j^* \geq v_1$$

Let the distribution $p^* = (p_1^*, \dots, p_m^*)$ be the one that minimizes the right side. Similarly, for all $j \in \{1, \dots, n\}$:

$$\sum_{i=1}^m p_i^* a_{ij} \leq v_2$$

Now consider the double sum $S = \sum_{i=1}^m \sum_{j=1}^n p_i^* a_{ij} q_j^*$. On one hand, we can sum over j first:

$$S = \sum_{i=1}^m p_i^* \left(\sum_{j=1}^n a_{ij} q_j^* \right)$$

Since all $p_i^* \geq 0$ and the term in the parentheses is $\geq v_1$ for all i :

$$S \geq \sum_{i=1}^m p_i^* (v_1) = v_1 \sum_{i=1}^m p_i^* = v_1 \cdot 1 = v_1$$

On the other hand, we can sum over i first:

$$S = \sum_{j=1}^n q_j^* \left(\sum_{i=1}^m p_i^* a_{ij} \right)$$

Since all $q_j^* \geq 0$ and the term in the parentheses is $\leq v_2$ for all j :

$$S \leq \sum_{j=1}^n q_j^* (v_2) = v_2 \sum_{j=1}^n q_j^* = v_2 \cdot 1 = v_2$$

Combining these two inequalities, we obtain:

$$v_1 \leq S \leq v_2 \implies v_1 \leq v_2$$

This algebraically proves the weak duality minimax inequality: $\max_q \min_i \sum_j a_{ij} q_j \leq \min_p \max_j \sum_i p_i a_{ij}$.

7. Chapter 2, Problem 7: Search

Lower and Upper Bounds on Expected Steps:

Upper Bound:

Any deterministic algorithm is effectively a special Las Vegas algorithm with no "randomness". We can directly use classic deterministic Binary Search. The number of steps for binary search in the worst case is $\lfloor \log_2 n \rfloor + 1$. Therefore, the upper bound on expected search steps is $O(\log n)$.

Lower Bound:

According to **Yao's Minimax Principle**, the expected cost of any randomized algorithm on worst-case inputs is bounded below by the expected cost of the best deterministic algorithm on a given probability distribution of inputs. Consider the following input distribution: the target key k is uniformly located at one of the n positions in the array (or in the $n + 1$ intervals between elements). For any deterministic search algorithm, its execution process can be represented as a decision tree. Since there are n possible valid locations in the array, this decision tree must have at least n leaves. In a binary tree with n leaves, the average depth lower bound is $\Omega(\log n)$. Thus, the expected number of steps for a deterministic algorithm over a uniform distribution is at least $\Omega(\log n)$. By Yao's principle, the expected number of steps for a Las Vegas randomized algorithm in the worst case must also be at least $\Omega(\log n)$.

In conclusion, the tight bound for the expected number of steps is $\Theta(\log n)$.

8. Chapter 2, Problem 8: Perfect Match

Proof that $C(n) = \Omega(n^2)$:

We use Yao's Minimax Principle to find the lower bound. We need to construct a probability distribution $\mathcal{D}$ over input graphs such that any deterministic algorithm determining if a perfect matching exists has an expected query count of $\Omega(n^2)$.

Distribution Construction:

Divide the n vertices evenly into two sets L and R , each of size $n/2$. Randomly generate a perfect matching M between L and R (there are $(n/2)!$ such matchings, drawn uniformly at random). Then uniformly at random pick one edge e^* from M . The input distribution $\mathcal{D}$ is defined as:

- With probability $1/2$, the input graph G is exactly the perfect matching M . The graph **has** a perfect matching.

- With probability $1/2$, the input graph G is $M \setminus \{e^*\}$. The graph **does not have** a perfect matching (two vertices have degree 0).

Analysis of Deterministic Algorithms:

For any deterministic query algorithm to correctly distinguish whether the input graph is M or $M \setminus \{e^*\}$ with absolute certainty, it must fundamentally confirm if those $n/2$ edges in M are intact. Since the hidden M is chosen uniformly among all $(n/2)!$ perfect matchings, when the algorithm makes its k -th query, assuming previous queries returned "No", the probability P that this query successfully hits an edge in M is the ratio of remaining possible edges. The total number of possible edges in the bipartite graph is $\approx n^2/4$, and M only has $n/2$ edges. For a random test, the success probability is roughly:

$$P \approx \frac{n/2}{n^2/4} = \frac{2}{n}$$

To verify (discover) one edge, the algorithm needs to test $\Omega(n)$ edges on average. To confirm the existence of the entire perfect matching's $n/2$ edges (or find the missing edge e^*), the algorithm essentially needs to find $\Theta(n)$ very sparse positive examples. By the logic of the "Coupon Collector's problem" or the sum of expectations, to locate these edges, the expected number of queries for a deterministic algorithm under distribution $\mathcal{D}$ is:

$$\text{Expected Queries} = \Omega\left(\frac{n}{2} \times n\right) = \Omega(n^2)$$

Therefore, by Yao's Minimax Principle, the expected number of queries $C(n)$ for a randomized algorithm in the worst case is $\Omega(n^2)$.

9. Chapter 19, Problem 7: $\alpha > n^2$

Proof: In the Local Connection Game, when the edge creation cost is $\alpha > n^2$, all Nash Equilibria must be trees, and the Price of Anarchy (PoA) is bounded by a constant.

1. Proof that all Nash equilibria are trees:

Assume there exists a graph G corresponding to a Nash equilibrium that contains a cycle. Then there must be an edge $e = (u, v)$ in the cycle paid for by some node (say, u). If node u decides to unilaterally deviate from its strategy by ****dropping (deleting)**** this edge e :

- u 's cost for buying edges decreases by α .
- Because e was originally in a cycle, deleting e leaves the graph G connected. In any connected graph with n nodes, the maximum distance (diameter) between any two nodes cannot exceed $n - 1$.

- Therefore, for the other $n - 1$ nodes in the network, u 's distance to them increases to at most $n - 1$ in the worst case.
- The total penalty increase for node u due to increased distances is at most $(n - 1) \times (n - 1) = (n - 1)^2$.

Given the condition $\alpha > n^2$, it is clear that $\alpha > n^2 > (n - 1)^2$. This means the cost saved by dropping $e(\alpha)$ is **strictly greater** than any potential distance penalty it incurs. Thus, node u has a strict incentive to unilaterally delete this edge. This contradicts the assumption that G is a Nash equilibrium. Therefore, no graph containing a cycle can be a Nash equilibrium. All Nash equilibrium graphs must be acyclic connected graphs, i.e., **trees**.

2. Proof that the Price of Anarchy is bounded by a constant:

Because edges are extremely expensive ($\alpha > n^2 > 2$), the social optimum graph (OPT) that minimizes Social Cost must be a **Star Graph**. The social cost consists of the total cost of purchasing edges plus the sum of distances between all pairs of nodes.

- **Cost of OPT:** A star graph contains $n - 1$ edges, with a cost of $\alpha(n - 1)$. The sum of distances is a constant multiple of n^2 (i.e., $\Theta(n^2)$). So, $Cost(OPT) = \alpha(n - 1) + \Theta(n^2)$.
- **Cost of any worst-case Nash equilibrium:** We know the worst Nash equilibrium is also a tree. Any tree with n nodes has exactly $n - 1$ edges. Thus, the total edge cost remains identical at $\alpha(n - 1)$. The worst-case sum of distances for any tree is a line graph, which has a total distance cost on the order of $\Theta(n^3)$. So, $Cost(NE) = \alpha(n - 1) + O(n^3)$.

Now we calculate the PoA. Since $\alpha > n^2$, we have $n^3 < \alpha n$:

$$PoA = \frac{\max Cost(NE)}{Cost(OPT)} \leq \frac{\alpha(n - 1) + O(n^3)}{\alpha(n - 1) + \Theta(n^2)} \leq \frac{\alpha(n - 1) + O(\alpha n)}{\alpha(n - 1)} = \frac{O(\alpha n)}{\Omega(\alpha n)} = O(1)$$

This proves that the cost of the worst Nash equilibrium is only larger than the socially optimal cost by a constant factor, meaning the Price of Anarchy is bounded by a constant.

10. Chapter 19, Problem 9: PoA

Proof: In the global connection game, let k be the number of players. The social cost of a strategy profile S is the total cost of all edges used in the network:

$$C(S) = \sum_{e \in \cup_i P_i} c_e$$

The cost paid by an individual player i is their equal share of the edges they use:

$$c_i(S) = \sum_{e \in P_i} \frac{c_e}{k_e}$$

where k_e is the number of players using edge e in profile S . Note that the social cost is exactly the sum of the players' individual costs: $C(S) = \sum_{i=1}^k c_i(S)$.

Let S be any Nash equilibrium and S^* be the social optimum. Since S is a Nash equilibrium, no player can decrease their cost by unilaterally deviating. In particular, player i cannot decrease their cost by switching to their optimal path P_i^* from S^* . If player i switches to P_i^* , the maximum cost they would pay is if they were the only player on those edges, which is $\sum_{e \in P_i^*} c_e$. Therefore, the equilibrium condition implies:

$$c_i(S) \leq \sum_{e \in P_i^*} c_e$$

Summing this inequality over all k players gives:

$$C(S) = \sum_{i=1}^k c_i(S) \leq \sum_{i=1}^k \sum_{e \in P_i^*} c_e$$

We can rewrite the double sum on the right by counting how many times each edge e in the optimal network is used. Let k_e^* be the number of players using edge e in S^* . Then:

$$\sum_{i=1}^k \sum_{e \in P_i^*} c_e = \sum_{e \in \cup_i P_i^*} c_e \cdot k_e^*$$

Since there are only k players in total, the maximum number of players that can use any edge is k (i.e., $k_e^* \leq k$). Thus:

$$\sum_{e \in \cup_i P_i^*} c_e \cdot k_e^* \leq k \sum_{e \in \cup_i P_i^*} c_e = k \cdot C(S^*)$$

Combining these inequalities gives $C(S) \leq k \cdot C(S^*)$. Hence, the Price of Anarchy (PoA), which is the ratio of the cost of the worst Nash equilibrium to the social optimum, can never exceed k .

11. Chapter 19, Problem 10: Potential Function

Proof: To show that this weighted generalization does not have an exact potential function, it suffices to construct a cycle of unilateral deviations where the sum of the

differences in the players' costs is not zero. If an exact potential function Φ existed, the sum of changes in Φ along any closed cycle would necessarily be zero, requiring the sum of the deviating players' cost changes to also equal zero.

Consider a game with two players, 1 and 2, with weights $w_1 = 1$ and $w_2 = 2$. Suppose they both want to travel from a common source s to a common sink t , and there are two parallel edges (paths) connecting s to t : Edge A with cost $c_A = 12$, and Edge B with cost $c_B = 12$.

Let's trace a cycle of four states and calculate the cost for the deviating player at each step. The cost for player i on edge e is $c_i = c_e \frac{w_i}{W_e}$, where W_e is the total weight on edge e .

1. **State 1:** (A, B) . Player 1 is on A , Player 2 is on B .
 - Player 1 pays $12 \times \frac{1}{1} = 12$.
 - Player 2 pays $12 \times \frac{2}{2} = 12$.
2. **Player 1 deviates to $B \rightarrow$ State 2:** (B, B) . Both are on B . Total weight on B is $W_B = 1 + 2 = 3$.
 - Player 1 pays $12 \times \frac{1}{3} = 4$.
 - Change in Player 1's cost: $\Delta_1 = 4 - 12 = -8$.
3. **Player 2 deviates to $A \rightarrow$ State 3:** (B, A) . Player 1 is on B , Player 2 is on A .
 - Player 2 pays $12 \times \frac{2}{2} = 12$.
 - Change in Player 2's cost (from State 2 where they paid $12 \times 2/3 = 8$): $\Delta_2 = 12 - 8 = +4$.
4. **Player 1 deviates to $A \rightarrow$ State 4:** (A, A) . Both are on A . Total weight on A is $W_A = 3$.
 - Player 1 pays $12 \times \frac{1}{3} = 4$.
 - Change in Player 1's cost (from State 3 where they paid 12): $\Delta_3 = 4 - 12 = -8$.
5. **Player 2 deviates to $B \rightarrow$ Return to State 1:** (A, B) . Player 1 is on A , Player 2 is on B .
 - Player 2 pays $12 \times \frac{2}{2} = 12$.
 - Change in Player 2's cost (from State 4 where they paid 8): $\Delta_4 = 12 - 8 = +4$.

The sum of the cost differences along this cycle of unilateral deviations is:

$$\sum \Delta c = -8 + 4 - 8 + 4 = -8$$

Since the sum of cost changes along a closed cycle is non-zero ($-8 \neq 0$), there cannot be an exact potential function for this game.

12. Chapter 19, Problem 13: Ordinal Potential Game

(a) Prove that if G is an ordinal potential game, then best response dynamics always converge to a Nash equilibrium.

Proof: In a finite game (a finite number of players and finite strategy sets for each), the total number of possible game states (strategy profiles) is finite. Whenever a player makes a best response (or any strictly improving move), their individual utility strictly increases, which means $u_i(S') - u_i(S) > 0$. By the definition of an ordinal potential game, the change in the potential function $\Phi(S) - \Phi(S')$ shares the same sign as $u_i(S') - u_i(S)$. Thus, $\Phi(S) - \Phi(S') > 0$, meaning $\Phi(S') < \Phi(S)$.

Therefore, every strictly improving move results in a strictly smaller value of the potential function Φ . Since the set of all possible game states is finite, the function Φ can only take on a finite set of values. A sequence of strictly decreasing values chosen from a finite set must eventually terminate. The sequence terminates when no player can make a strictly improving move to decrease Φ further. A state where no player can make an improving move is precisely the definition of a Nash equilibrium. Thus, best response dynamics must always converge to a Nash equilibrium.

(b) Prove that if best response dynamics always converge, then G is an ordinal potential game.

Proof: Consider a directed state graph where each vertex represents a strategy profile (game state) S . We add a directed edge from S to S' if S' can be reached from S by a single player making a strictly improving move (a unilateral deviation where $u_i(S') > u_i(S)$).

The assumption that "from any starting configuration, best response dynamics always converge to an equilibrium" implies that there are no infinite paths in this directed graph. Since the graph is finite, the absence of infinite paths is equivalent to the graph having no directed cycles (it is a Directed Acyclic Graph, or DAG).

Because the state graph is a DAG, we can perform a topological sort on its vertices. A topological sort assigns an integer index to each vertex such that if there is a directed

path from S to S' , the index of S is strictly greater than the index of S' . Let $\Phi(S)$ be this topological index for state S .

- If a move from S to S' is a strictly improving move ($u_i(S') - u_i(S) > 0$), there is a directed edge $S \rightarrow S'$. By the property of the topological sort, $\Phi(S) > \Phi(S')$, so $\Phi(S) - \Phi(S') > 0$. The signs match.
- If a move from S to S' is a strictly worsening move ($u_i(S') - u_i(S) < 0$), then the reverse move $S' \rightarrow S$ is a strictly improving move. This means there is a directed edge $S' \rightarrow S$, so $\Phi(S') > \Phi(S)$, and thus $\Phi(S) - \Phi(S') < 0$. The signs match.
- If a move from S to S' is a neutral move ($u_i(S') - u_i(S) = 0$), we can assign them the same potential value $\Phi(S) = \Phi(S')$ by grouping mutually reachable neutral states into equivalence classes prior to the topological sort. Then $\Phi(S) - \Phi(S') = 0$, and the signs match (both are 0).

Since we have constructed a real-valued function $\Phi(S)$ such that the sign of $\Phi(S) - \Phi(S')$ is always the same as the sign of $u_i(S') - u_i(S)$ for any unilateral deviation, the game G is an ordinal potential game.